

Lab 13 - Expected output reference

Pod IPs, ages and ReplicaSet hashes are runtime values and will differ.

How to read this file.

>

- Everything under Phase 1 is the stock, no-controller path (README Steps 1-8).

No curl transcript of routed traffic appears there, because none is produced.

- Everything under Phase 2 is the **expected result of the documented optional**

path in README Appendix C - the pinned Traefik install. It is derived from that

chart's own rendered manifests and the project's documentation, not from a

recording of your cluster. Run Appendix C and compare. If your output differs, your

output is the truth.

Phase 1 - stock cluster, no controller (MODE 1)

bash verification/checks.sh - clean run on a stock kind cluster

```
== Lab 13 verification: Ingress resources and HTTP routing ==
PASS namespace kcna-lab13 exists
PASS ConfigMap catalog-content matches data/catalog-index.html byte-for-byte
PASS all 3 backend Deployments are Available
PASS catalog-svc, checkout-svc and route-fallback-svc all have endpoints
PASS backend catalog-svc answers directly (Ingress bypassed)
PASS backend checkout-svc answers directly (Ingress bypassed)
PASS backend route-fallback-svc answers directly (Ingress bypassed)
PASS ingress meridian-shop exists with apiVersion networking.k8s.io/v1
PASS ingress rules match data/routing-table.csv exactly
PASS every pathType is one of Exact / Prefix / ImplementationSpecific
PASS ingress meridian-shop sets ingressClassName (meridian-edge)
PASS ingress meridian-shop declares a defaultBackend (route-fallback-svc:80)
PASS ingress meridian-shop-tls declares tls[0].secretName
PASS TLS Secret meridian-shop-tls-cert is type kubernetes.io/tls with 2 keys
NOTE MODE 1 (declaration-only): no IngressClass exists on this cluster.
PASS DECISIVE: 0 IngressClasses exist, so no controller can claim the Ingress
PASS DECISIVE: no Ready controller Pod in namespace kcna-lab13-ingress
NOTE CORROBORATING: .status.loadBalancer is empty (ADDRESS column blank). Consistent with the
decisive checks - on its own it would prove nothing.
SKIP live HTTP routing not exercised: this cluster has no ingress controller (MODE 1)
NOTE The Ingress objects are verified as CORRECT DECLARATIONS only.
NOTE README Appendix C installs a pinned controller and switches this script into MODE 2.
-- 16 passed, 0 failed, 1 skipped --
```

Exit status 0.

Two deliberate design choices in that output:

Line	Why it is that severity
The blank ADDRESS is a NOTE, not a PASS	It is corroboration, not evidence. A controller behind a ClusterIP Service routes correctly with ADDRESS blank forever, so the script never scores it
Routing is SKIP, not FAIL	The script did not test routing, so it asserts neither success nor failure. "Could not test" is never converted into "failed"

Phase 2 - after README Appendix C (MODE 2)

Expected, not observed. Produced by the documented commands; verify on your cluster.

Controller install and readiness

```
$ helm install meridian-edge traefik \
  --repo https://traefik.github.io/charts --version 41.4.0 \
  --namespace kcna-lab13-ingress --skip-crds \
  -f data/traefik-lab-values.yaml --wait --timeout 5m
NAME: meridian-edge
LAST DEPLOYED: ...
NAMESPACE: kcna-lab13-ingress
STATUS: deployed
REVISION: 1

$ kubectl rollout status deployment/meridian-edge-traefik -n kcna-lab13-ingress --timeout=180s
deployment "meridian-edge-traefik" successfully rolled out

$ kubectl get pods -n kcna-lab13-ingress
NAME                                READY   STATUS    RESTARTS   AGE
meridian-edge-traefik-...           1/1     Running   0           40s

$ kubectl get deployment meridian-edge-traefik -n kcna-lab13-ingress \
  -o jsonpath='{.spec.template.spec.containers[0].image}'
docker.io/traefik:v3.7.12

$ kubectl get ingressclass
NAME            CONTROLLER          PARAMETERS   AGE
meridian-edge   traefik.io/ingress-controller  <none>       45s
```

The counter-example that matters

```
$ kubectl get ingress -n kcna-lab13
NAME            CLASS          HOSTS
meridian-shop   meridian-edge  shop.meridianfreight.internal,tracking.meridianfreight.
internal        80             20m
meridian-shop-tls meridian-edge  secure.meridianfreight.internal
80, 443        16m
```

A controller is installed, is Ready and is about to route every rule correctly - and ADDRESS is still blank, because the controller's Service is a ClusterIP and providers.kubernetesIngress.publishedService is disabled. This is the concrete disproof of "blank ADDRESS means no controller".

Routing through kubectl port-forward

```
$ kubectl port-forward -n kcna-lab13 ingress deployment/meridian-edge-traefik 18080:8000 &
Forwarding from 127.0.0.1:18080 -> 8000
Forwarding from [::1]:18080 -> 8000
```

Request	Rule that should win	Expected body
Host: shop... /catalog	Prefix /catalog	catalog page
Host: shop... /catalog/containers/20ft	Prefix /catalog	catalog page
Host: shop... /catalogue	none - element boundary	no ingress rule matched - default backend :: kcna-lab13
Host: shop... /checkout/status	Exact /checkout/status	checkout status OK :: kcna-lab13
Host: shop... /checkout/status/	none - Exact rejects the trailing slash	no ingress rule matched - default backend :: kcna-lab13
Host: tracking... /	Prefix / on the second host	catalog page
Host: unmapped... /	none	no ingress rule matched - default backend :: kcna-lab13

```
$ curl -sS -H 'Host: shop.meridianfreight.internal' http://127.0.0.1:18080/catalog | head -5
<!DOCTYPE html>
<html lang="en">
<head><meta charset="utf-8"><title>Meridian Freight - Container Catalog</title></head>
<body>
<h1>Meridian Freight Pte Ltd</h1>

$ curl -sS -H 'Host: shop.meridianfreight.internal' http://127.0.0.1:18080/catalogue
no ingress rule matched - default backend :: kcna-lab13

$ curl -sS -H 'Host: shop.meridianfreight.internal' http://127.0.0.1:18080/checkout/status
checkout status OK :: kcna-lab13

$ curl -sS -H 'Host: shop.meridianfreight.internal' http://127.0.0.1:18080/checkout/status/
no ingress rule matched - default backend :: kcna-lab13

$ curl -sS -H 'Host: unmapped.meridianfreight.internal' http://127.0.0.1:18080/
no ingress rule matched - default backend :: kcna-lab13
```

The /catalogue result depends on `providers.kubernetesIngress.strictPrefixMatching`:

true, which data/traefik-lab-values.yaml sets. Traefik's default prefix matching is

character-by-character, which would wrongly match /catalogue. If you see the catalog

page there, check that value first.

bash verification/checks.sh - MODE 2

```
NOTE  MODE 2 (controller present): 1 IngressClass(es) found on this cluster.
PASS  DECISIVE: 1 IngressClass(es) exist, so a controller is installed
PASS  IngressClass 'meridian-edge' exists and names controller 'traefik.io/ingress-controller',
      so meridian-shop is claimable
PASS  DECISIVE: 1 Ready controller Pod(s) in namespace kcna-lab13-ingress
NOTE  CORROBORATING: ADDRESS is blank. That is NOT a failure - a ClusterIP controller reached
      by port-forward publishes no address and still routes.
PASS  port-forward 127.0.0.1:18080 -> kcna-lab13-ingress/meridian-edge-traefik:8000 is up
PASS  ROUTED shop.../catalog (Prefix) -> catalog-svc
PASS  ROUTED shop.../catalog/containers/20ft (Prefix, deeper path) -> catalog-svc
PASS  ROUTED shop.../catalogue does NOT match Prefix /catalog (element boundary) ->
      defaultBackend
PASS  ROUTED shop.../checkout/status (Exact) -> checkout-svc
PASS  ROUTED shop.../checkout/status/ trailing slash breaks Exact -> defaultBackend
PASS  ROUTED tracking.../ (host fan-out, Prefix /) -> catalog-svc
PASS  ROUTED unmapped host -> defaultBackend route-fallback-svc
-- 25 passed, 0 failed, 0 skipped --
```

Useful diagnostic failures the script can produce in MODE 2:

```
FAIL  an IngressClass exists but none is named 'meridian-edge': meridian-shop.spec.
      ingressClassName is a forward reference to a missing IngressClass
SKIP  live HTTP routing not exercised: controller Deployment kcna-lab13-ingress/meridian-edge-
      traefik not found (override ING_NS / ING_DEPLOY)
SKIP  live HTTP routing not exercised: port-forward to kcna-lab13-ingress/meridian-edge-
      traefik:8000 did not come up within 30s (is LOCAL_PORT 18080 free?)
```

Static validation of the vendored controller manifest

```
$ kubeconform -strict -summary -kubernetes-version 1.31.0 data/traefik-41.4.0-rendered.yaml
Summary: 6 resources found in 1 file - Valid: 6, Invalid: 0, Errors: 0, Skipped: 0
```

Skipped: 0 matters: no CRD schemas are needed, because the install uses --skip-crds and disables providers.kubernetesCRD.

Key intermediate outputs (Phase 1)

The Ingress list (Step 4 and Step 7)

NAME	CLASS	HOSTS
ADDRESS	PORTS	AGE
meridian-shop	meridian-edge	shop.meridianfreight.internal,tracking.meridianfreight.
internal	80	4m
meridian-shop-tls	meridian-edge	secure.meridianfreight.internal
	80, 443	6s

Column	Value here	Reading	Strength
CLASS	meridian-edge	The object names a class. <none> would mean its fate is controller-dependent	Strong

Column	Value here	Reading	Strength
ADDRESS	blank	Nothing has written .status.loadBalancer.ingress[]	Weak - see Phase 2, where routing works with this still blank
PORTS	80 / 80, 443	443 appears purely because the second object has a tls block. It is not evidence that TLS is being served	Strong

kubectl describe ingress meridian-shop

```

Name:          meridian-shop
Namespace:     kcna-lab13
Address:
Ingress Class: meridian-edge
Default backend: route-fallback-svc:80 (10.244.0.44:5678)
Rules:
  Host            Path  Backends
  ----            -
  shop.meridianfreight.internal
                                /catalog      catalog-svc:80 (10.244.0.42:8080)
                                /checkout/status checkout-svc:80 (10.244.0.43:5678)
  tracking.meridianfreight.internal
                                /              catalog-svc:80 (10.244.0.42:8080)
Annotations:    <none>
Events:         <none>

```

The parenthesised Pod addresses come from kubectl resolving each Service's endpoints. Seeing real IPs there proves the backends are fine, which isolates any fault to the controller layer. The contrasting failure looks like:

```

shop.meridianfreight.internal
                                /catalog      catalog-svc:80 (<error: endpoints "catalog-
svc" not found>)

```

Events: <none> is a weak signal on its own: some controllers emit sync events, many emit none in normal operation.

The rules dumped for comparison against data/routing-table.csv

```

shop.meridianfreight.internal,/catalog,Prefix,catalog-svc,80
shop.meridianfreight.internal,/checkout/status,Exact,checkout-svc,80
tracking.meridianfreight.internal,/,Prefix,catalog-svc,80

```

Byte-identical to the CSV body. This is the whole verification of routing *intent* that can be performed without a controller.

Controller determination (Step 8)

```

$ kubectl get ingressclass
No resources found                                <- DECISIVE

$ kubectl get pods --all-namespaces -l app.kubernetes.io/component=controller
No resources found                                <- DECISIVE

$ kubectl get ingress meridian-shop -n kcna-lab13 -o jsonpath='{.status.loadBalancer}'
{}
                                                <- corroborating only

```

Two decisive checks plus one corroboration - not "three independent confirmations of equal weight".

Failure-injection reference (Section 6)

Loud failure - invalid pathType:

```
The Ingress "meridian-shop-broken" is invalid: spec.rules[0].http.paths[0].pathType:
Unsupported value: "prefix": supported values: "Exact", "ImplementationSpecific", "Prefix"
```

The same file passes static validation:

```
$ kubeconform -strict -summary manifests/60-ingress-invalid-pathtype.yaml
Summary: 1 resource found parsing 1 file - Valid: 1, Invalid: 0, Errors: 0, Skipped: 0
```

Silent failure - created successfully with no class:

NAME	CLASS	HOSTS	ADDRESS	PORTS	AGE
meridian-shop-broken	<none>	shop.meridianfreight.internal		80	12s

```
Ingress Class: <none>
Address:
Events: <none>
```

No error is emitted anywhere. The only evidence is the absence of evidence.

Environment caveats recorded for this lab

Caveat	Observed effect	Why
Stock kind has no ingress controller	Ingress objects are created and correct but route nothing; ADDRESS blank, Events: <none>	The Ingress API is served by the API server; routing requires a separately installed controller. Phase 1 never shows routed-traffic output it did not produce
No IngressClass exists	kubectl get ingressclass → No resources found. ingressClassName: meridian-edge is a forward reference to a non-existent object	IngressClass objects are installed by controllers. Naming a non-existent class is legal and silent. Appendix C makes the reference resolve
A blank ADDRESS is not proof of anything	In Phase 2 a working controller routes every rule with ADDRESS permanently blank	ADDRESS renders <code>.status.loadBalancer.ingress[]</code> , which a ClusterIP controller with publishing disabled never writes. Use IngressClass + controller Pod as the decisive checks
A class-less Ingress is controller-dependent	Created, CLASS <none>, never reconciled *on this cluster*, no error	With no controller there is nothing to claim it. With a controller, adoption depends on that controller: is-default-class is one mechanism, and some controllers (Traefik among them) document claiming class-less Ingresses on their own terms
The ingress-nginx *controller project* was retired in March 2026	Not used or recommended by this lab	https://kubernetes.io/blog/2025/11/11/ingress-nginx-retirement/ . The networking.k8s.io/v1 Ingress API is unaffected, and so is the NGINX web server image <code>nginx:1.27-alpine</code> used as the catalog backend here - different thing, same name
ingress-ready=true and extraPortMappings are create-time only	A running kind cluster cannot be retrofitted with host port publishing	Requires kind create cluster --config (Appendix A). This does not block live routing: kubectl port-forward reaches the controller on any running cluster, which is what Appendix C uses
Traefik's default prefix matching is not spec-conformant	Without strictPrefixMatching, /catalogue would match Prefix: /catalog	Documented at https://doc.traefik.io/traefik/reference/install-configuration/providers/kubernetes/kubernetes-ingress/ . <code>data/traefik-lab-values.yaml</code> sets it true
The TLS Secret can be missing without error	meridian-shop-tls applies cleanly before the Secret exists	The Ingress does not validate the reference; a live controller would log a warning and fall back to its default certificate
hashicorp/http-echo:1.0 must be pullable	ImagePullBackOff for checkout and route-fallback in an air-gapped room	Substitute <code>nginx:1.27-alpine</code> on containerPort: 8080 with its own ConfigMap; the routing declarations under test are unchanged
docker.io/traefik:v3.7.12 must be pullable (Appendix C only)	Controller Pod stuck in ImagePullBackOff; checks.sh stays in MODE 1 and reports SKIP	Appendix C is optional. Pre-pull and kind load docker-image <code>docker.io/traefik:v3.7.12</code> in an air-gapped room